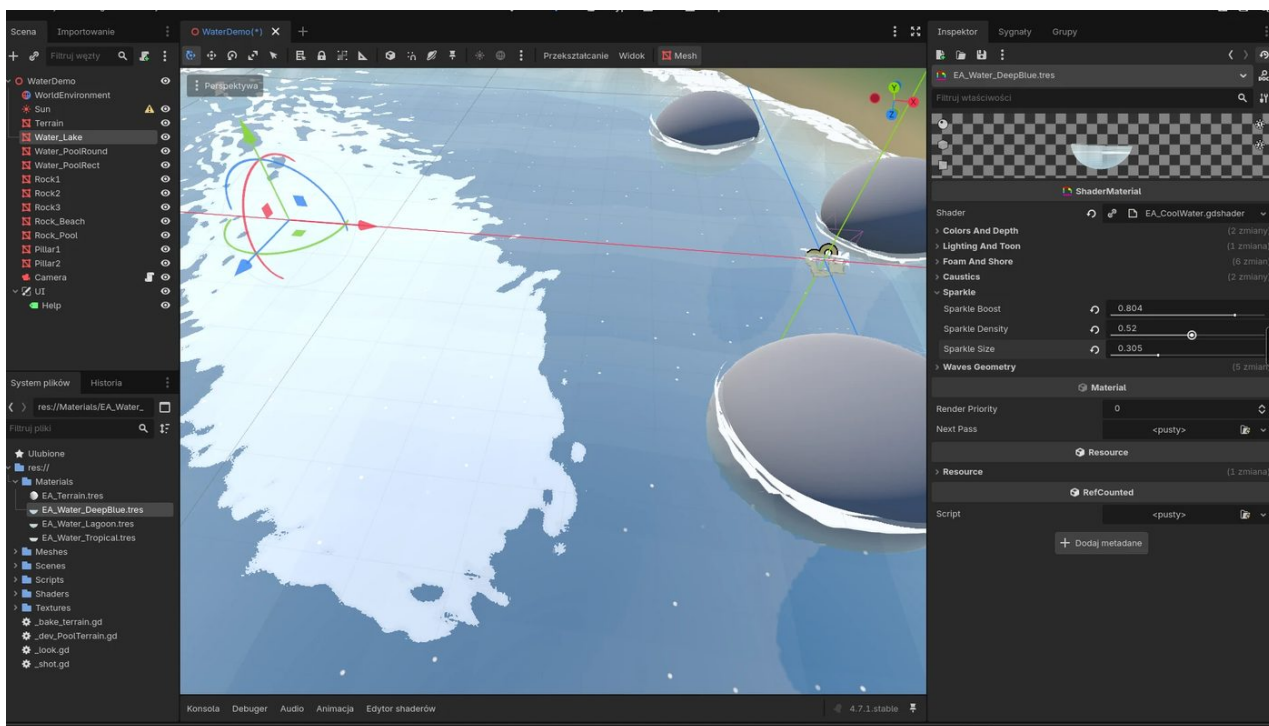


EmacEArt Cool Water — water shader for Godot 4

Stylized toon water for games: lakes, ponds, pools, roadside puddles. The pack contains one shader file, three ready materials and a demo scene.

The water goes on an ordinary plane placed over a dip in the terrain. It needs no textures, no extra geometry and no masks of its own. The shader measures how far the ground sits below the surface, and derives the water colour, the shoreline foam, the light patterns on the bottom and the glints on the surface from that single number.



Demo scene open in the editor

This is Scenes/WaterDemo.tscn the moment it opens in the editor. The terrain is a saved mesh, so the scene is finished before you press F5.

Requirements and setup

Godot **4.7 or newer**. In the project manager: **Import**, point at project.godot, **Import & Edit**. Then press F5, or open Scenes/WaterDemo.tscn and press F6.

UWAGA. The renderer must be Forward+ or Mobile. On the Compatibility renderer the water will not draw correctly. Godot stores scene depth in a different format there, and this whole shader is built on reading the depth buffer.

Demo scene controls

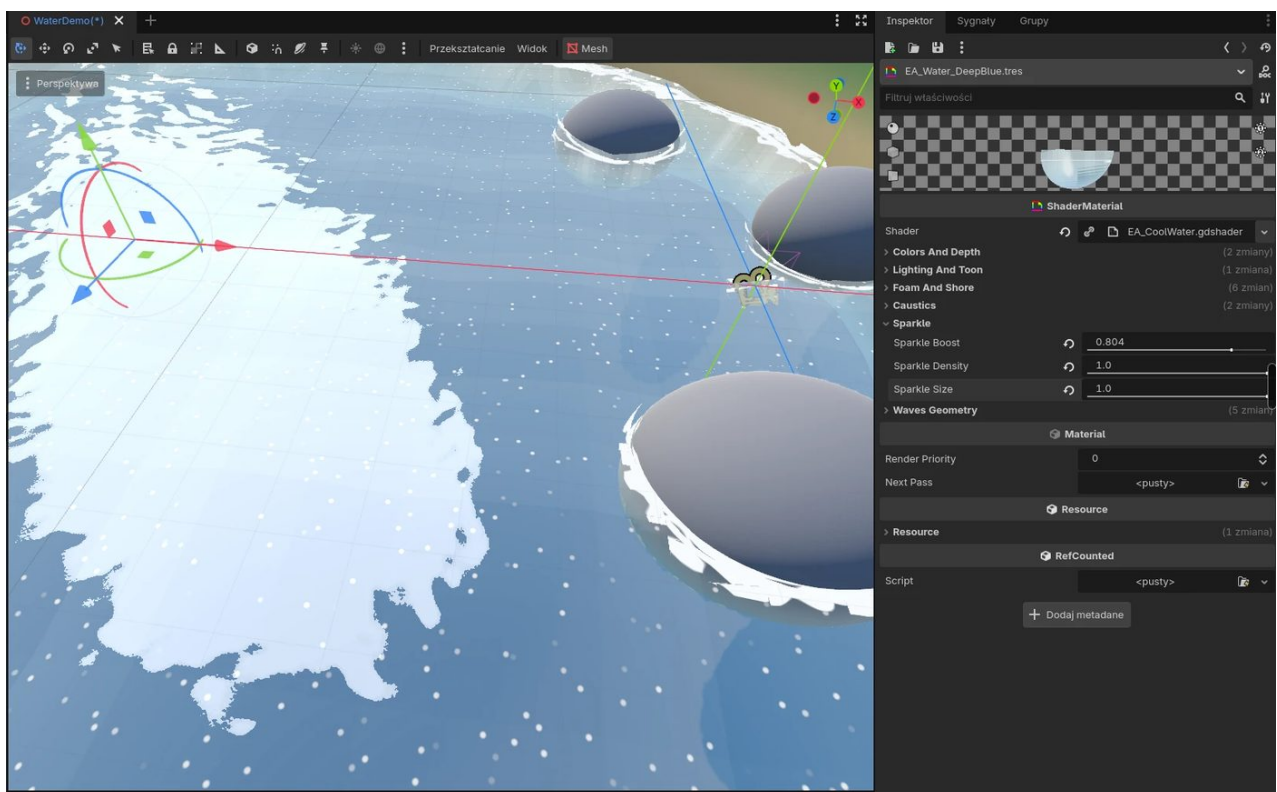
Input	Action
LMB + drag	orbit the camera around the target
RMB + drag	pan the target
Mouse wheel	zoom
1 / 2 / 3	overview / beach level / both pools
Space	freeze time — waves, foam and caustics stop
H	hide and show the slider panel together with the help text

Press **H** before taking screenshots — it hides the panel and the help text.

Where to change parameters

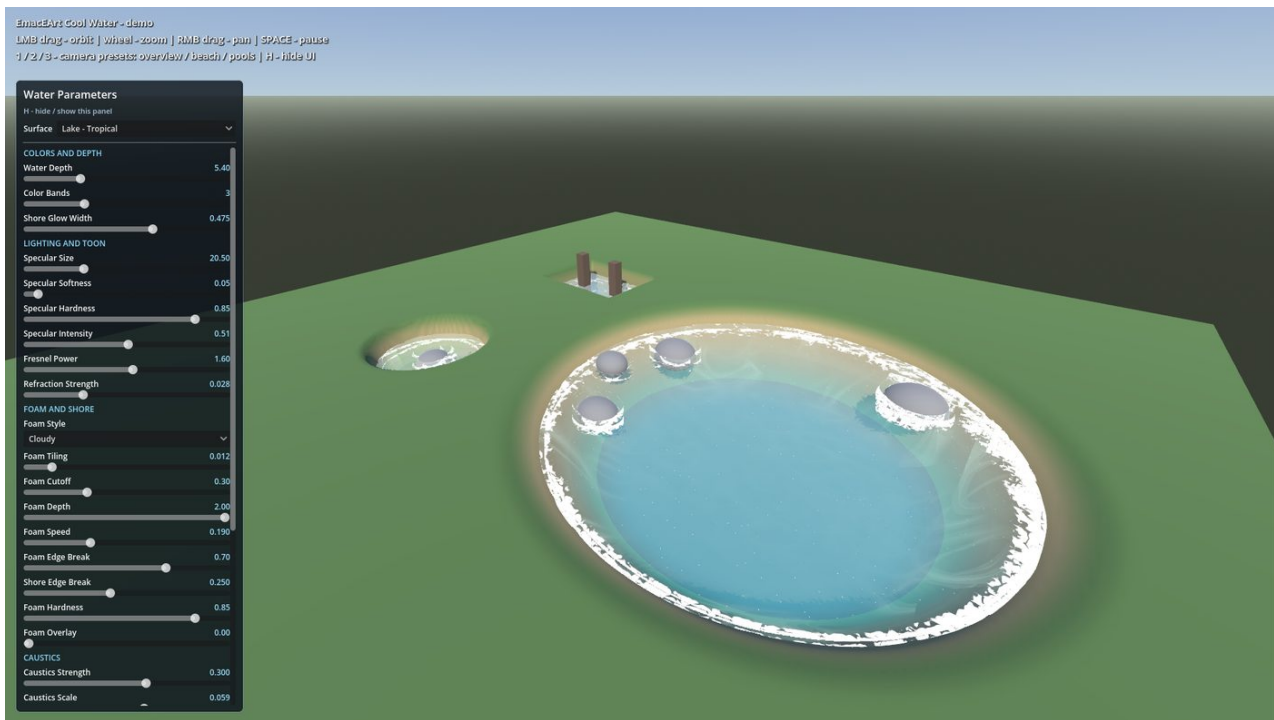
Two places, whichever suits you.

In the editor: select a water surface in the scene tree, then Inspector → Material Override → Shader Parameters. Changes are saved into the material file.



Water material in the Inspector, close view of the surface

In the running scene: the slider panel on the left of the screen. Pick a surface at the top of the panel — lake, round pool or rectangular pool — and drag the sliders. The water reacts instantly.



Slider panel in the running demo

Panel settings live only until you close the game. The .tres material files are never written to, so you can experiment without any risk to the project.

How the water knows where it is shallow

Everything comes from one number: **the vertical distance between the surface and the ground below it, in metres.**

The shader reads scene depth from the buffer, reconstructs the ground position in world space and subtracts it from the height of the surface. From that distance it derives the colour, the shore glow, the reach of the foam and the brightness of the caustics. Setting up a body of water therefore comes down to shaping the ground and placing a plane above it.

The distance is measured along world vertical, not along the camera ray. That keeps the colour bands and the foam in place when the camera orbits or tilts.

Where the terrain rises above the water line — a rock, a bridge pile, a small island — the ground ends up higher than the surface. The shader discards those pixels. Without that, the water would draw a visible ring around every such object.

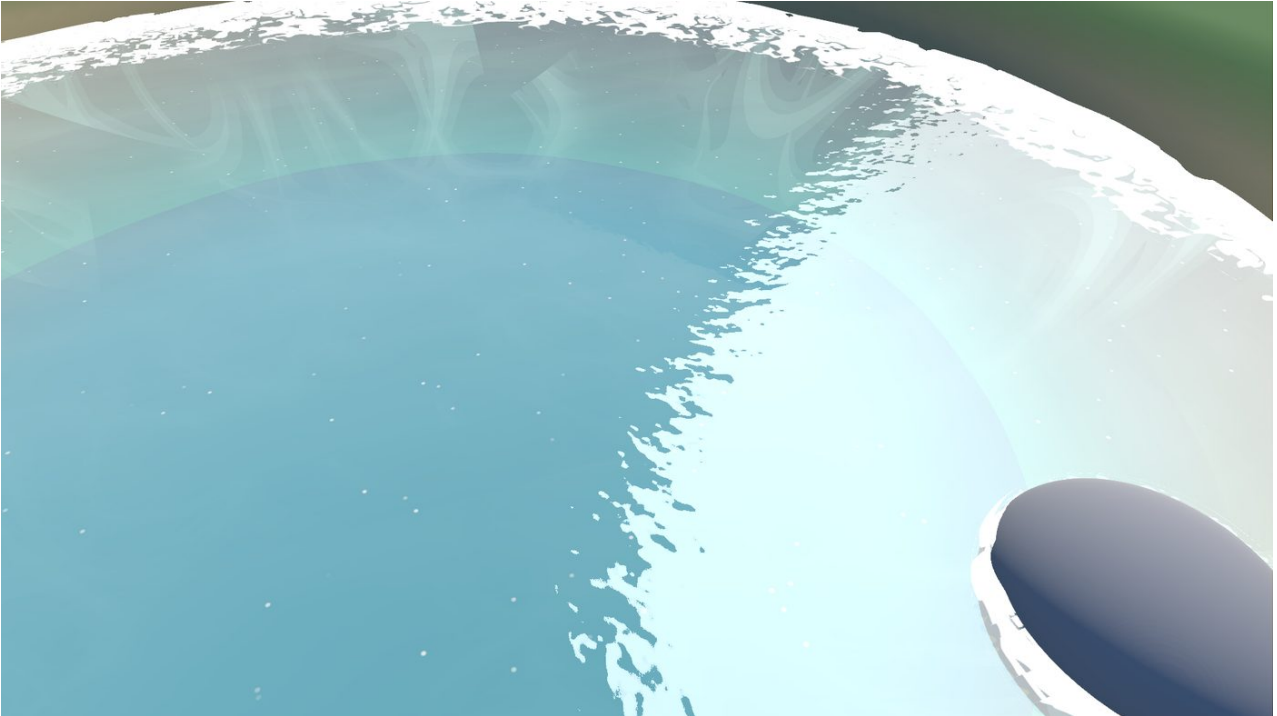
Colour and depth

The distance to the ground is split into bands. `water_depth` sets the depth at which the water reaches the full `deep_color`, and `color_bands` sets how many steps the way up from `shallow_color` is divided into. One band gives a flat colour, eight give a smooth gradient.

The alpha channel of both colours works independently of RGB: the shader blends the shallow transparency with the deep transparency using the same number it uses for the hue.

The shore glow is separate — a band of `shore_glow_color` along the line where the water meets land. `shore_glow_width` gives its width as a fraction of `water_depth`: at 0.15 the glow reaches 15 percent of

the full depth.



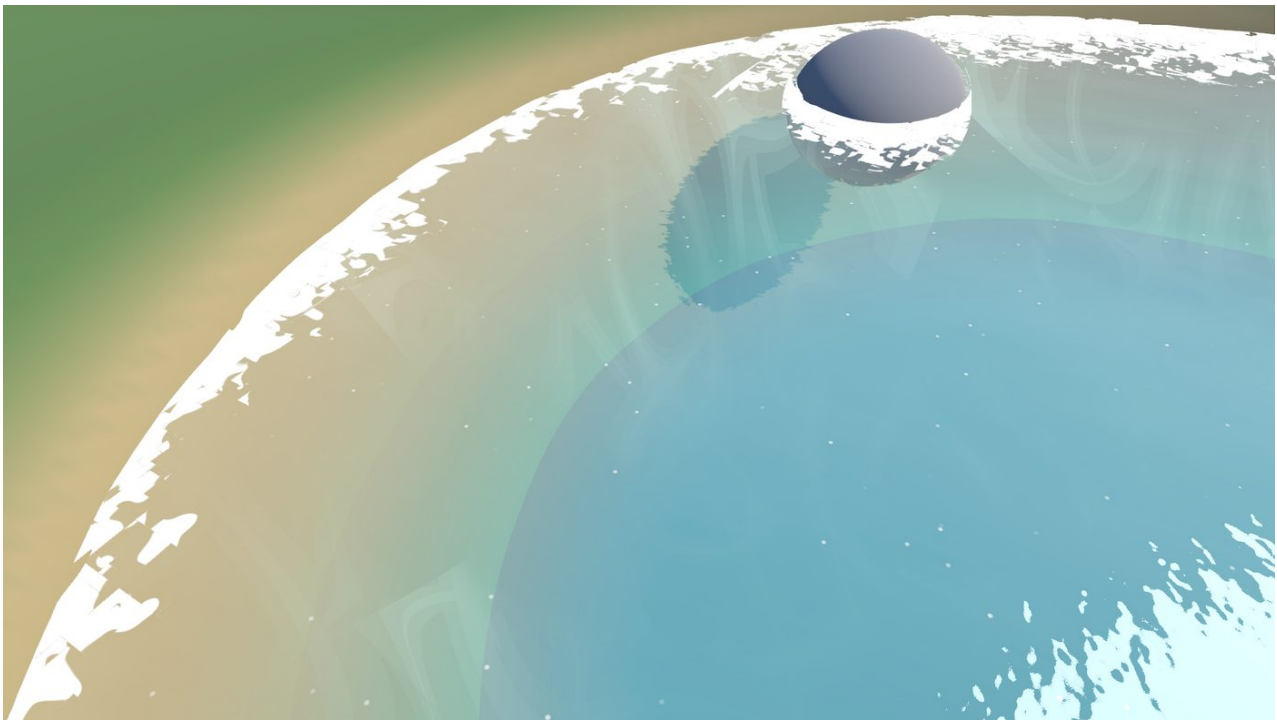
The Tropical material with color_bands set to 3. Three distinct colour steps are visible: turquoise near the shore, a lighter band in the middle and a darker blue over the deepest part. The white dots on the surface are the sparkle layer.

Foam

Foam has two sources, independent of each other. It shows wherever either one wins.

Shoreline foam builds itself from the shape of the ground. The shader samples depth at four points next to the pixel being shaded (8 pixels out on the diagonals) and takes the shallowest result. Where the ground comes close to the surface it lays down a band of foam: first a narrow solid rim right at the edge, then ragged tongues broken up by the pattern. foam_depth sets how many metres from the shore the whole mechanism covers, and foam_cutoff how far into that range the foam actually reaches.

A rock or a bridge pile sticking out of the water gets a foam ring from this with no extra work. The ground next to it is simply shallow.



The boulder has no extra mask and no painted vertex colour. The full ring around it and the ragged band along the beach on the left both come out of the same depth measurement.

Painted foam is driven by the red vertex colour channel — see the section on painted zones below.

Shape and edges

`foam_edge_break` chews the foam band into strips. At 0 both edges — inner and outer — stay even; at 1 the band breaks into irregular patches with gaps. The shader offsets the distance-to-shore field with continuous noise, so both edges get the same irregularity at once. Range 0–1, default 0.6.

`shore_edge_break` disturbs the waterline itself. The pixel discarding described above (ground higher than the surface) is mathematically exact, so the edge of the water runs along a smooth arc and the foam reaching it ends just as smoothly. This slider is the height of that cut-off threshold in metres, with noise added to it — the waterline then runs into the shore unevenly, like wet sand. Range 0–0.6, default 0.25.

`foam_hardness` sets how abruptly the foam mask ends. It acts on both masks at once — shoreline and painted — because both go through the same thresholding function. At 0 you get a soft falloff, at 1 the edge is cut as if by a stencil, with a one-pixel blend so the line does not staircase along a diagonal shore. This is the mechanism behind the stylized, toon look of the foam rather than a blurred one. Range 0–1, default 0.8.

Pattern

`foam_style` picks the foam pattern. The default is **Cellular**.

Mode	What it gives
Texture	pattern from the foam_texture sampler
Cloudy	soft streaks from three noise layers
Blobs	large solid patches with a hard edge
Scatter	fine, dense speckles
Cellular	voronoi cells with bright interiors

foam_tiling does not state density directly. The slider runs 0–0.1 (default 0.02) and the shader converts that value into pattern repeats per metre. A lower value gives very large shapes, a higher one a finer pattern.

foam_overlay sets how the foam is mixed with the water beneath it. At 0 the foam is laid down as flat foam_color, always the same regardless of what is underneath. At 1 the mix goes through overlay: the water colour shows through, so foam over the deep end comes out cooler and foam over the shallows brighter. Range 0–1, default 0.0.

Wazne. With a white foam_color over bright water both modes look practically identical — overlaying white onto a light colour stays close to white. The difference shows up with foam tinted a colour other than white, or over a dark deep end. That follows from the overlay maths and is not a shader fault.

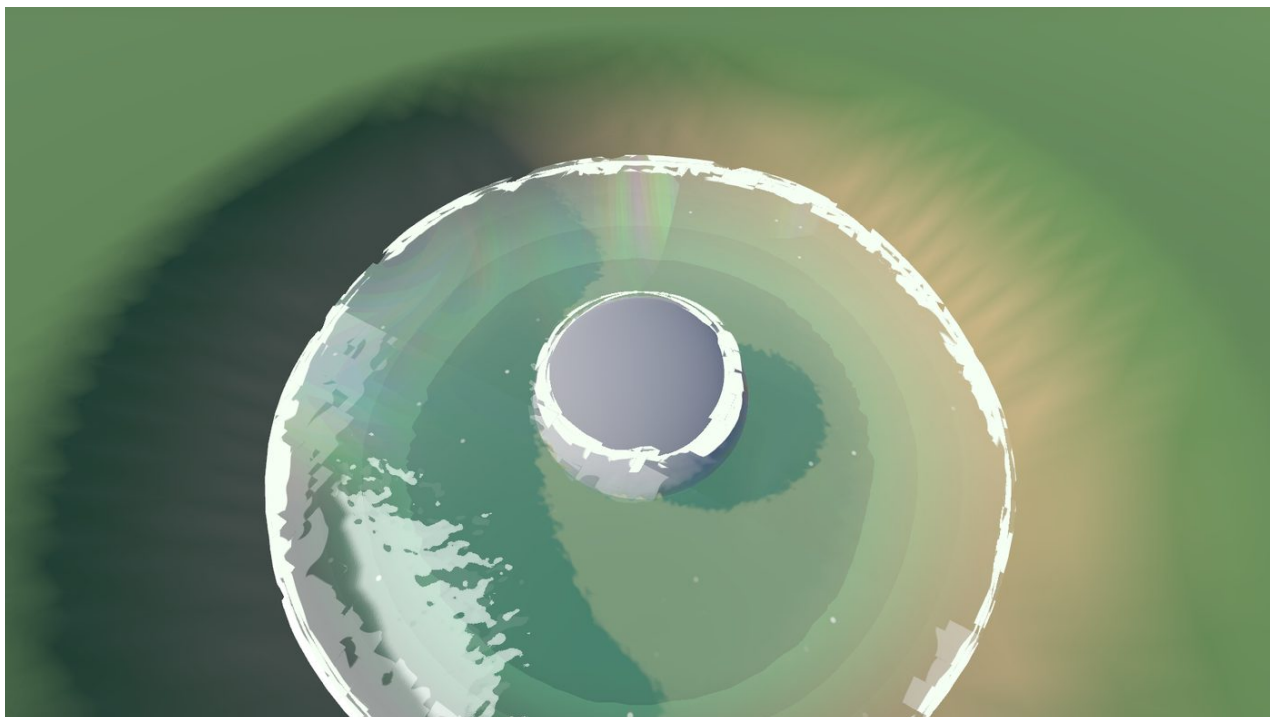
Caustics

A network of bright light veins on the bottom. It comes from four wave sets running at irrational angles, pre-distorted with noise — straight ridges bend into irregular streaks instead of settling into a grid. On top of that a slow noise mask cuts parts of the surface out entirely. Without it the effect would cover the bottom in an even layer.

The veins are sampled **at the ground position**, not the surface. The patches of light therefore lie on the terrain and creep over rocks under the water instead of sliding across the surface.

The caustic colour picks itself: the shader blends shallow_color with shore_glow_color and brightens the result. caustics_tint, white by default, only modulates that. The light is applied in screen mode, so it passes through the water colour rather than sitting on it as a layer.

chromatic_aberration enables three offset samples, one per colour channel. A rainbow fringe then appears along the edges of the veins. It costs three times as much as the version with it off. In the shader itself it defaults to on (true). Of the three demo materials it is enabled only on the lagoon — on the lake and the rectangular pool it is off, so the rainbow fringes do not take over the main shot.



A shot of the lagoon, the only material in the demo with chromatic aberration enabled — the rainbow streaks run along the bright veins on the left. The depth bands and the foam going around the rock are visible here as well.

Highlight and sparkle

The highlight is a toon one with four independent knobs: `specular_size` (how tight, range 5–60, default 24), `specular_softness` (how soft the edge, 0 being a hard cut; range 0–1, default 0.25), `specular_hardness` (a second flattening of whatever `specular_softness` left) and `specular_intensity` (brightness, range 0–1, default 0.5). The colour comes from `specular_color`, white by default.

The normal used for the highlight is sampled from the normal map at a raised frequency. Without that, a flat surface would produce one large blob.

`specular_hardness` acts on the gradient `specular_softness` left behind. At 0 the gradient stays as it is; at 1 every pixel that lit up at all goes to the same full brightness. At high values the reflection loses the transition between centre and edge — what remains is flat white chips of uniform brightness scattered across the surface, instead of one spread blob. Range 0–1, default 0.85.

The highlight is computed in the `light()` function, so it responds to every light in the scene, not just the sun.

Sparkle is a separate layer — tiny flickering points scattered over the whole surface, independent of the highlight. Every grid cell holds one candidate glint, but only some cells light up and each flickers in its own phase. That gives irregular twinkling rather than a grid of dots. Where there is already foam the glints do not show — the foam is white anyway.

Parameter	What it does	Range	Default
sparkle_boost	brightness of the whole layer; at 0 the mechanism is skipped	0–1	0.0
sparkle_density	how many glints appear and how close together	0–1	0.5
sparkle_size	size of a single glint	0–1	0.35

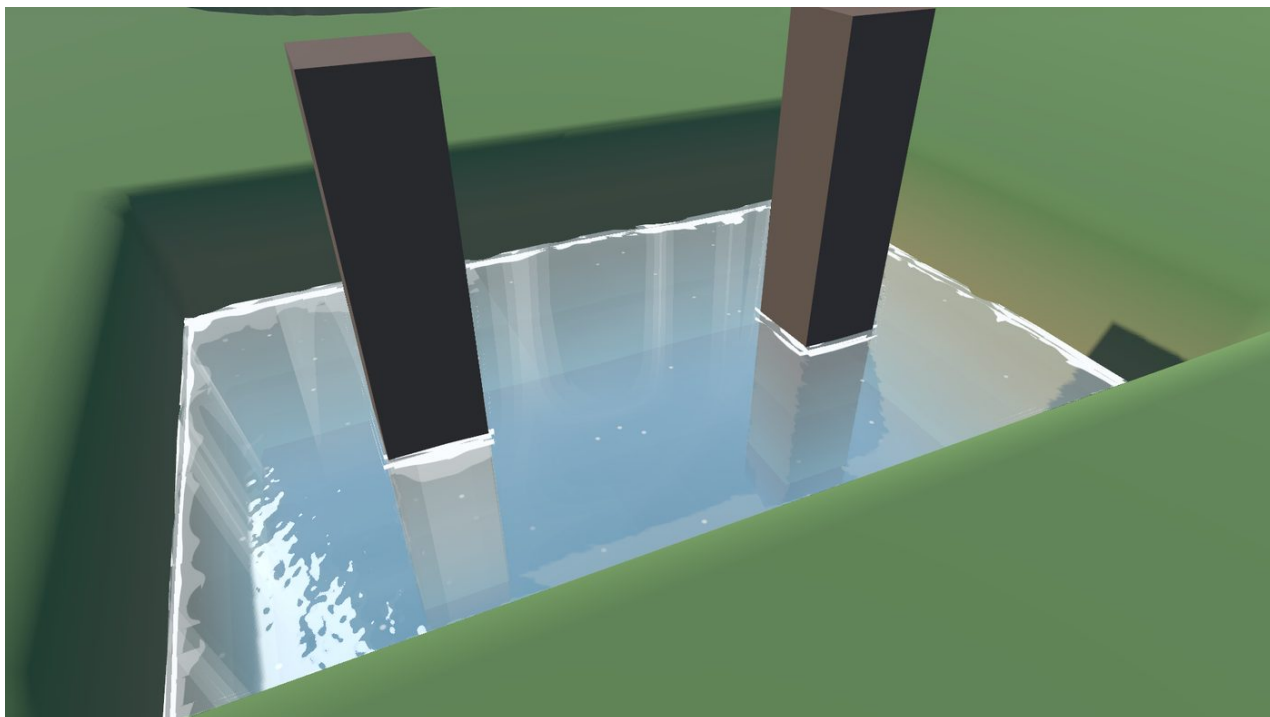


Glints scattered across the whole surface, far away from the highlight as well. Each of the four boulders has a foam ring, and along the edge of the frame there is the foam band following the shoreline.

Refraction

The image beneath the water is offset according to the normal map. `refraction_strength` sets how strongly. The deeper it gets, the less of the bottom shows, because the water colour gradually covers it.

Refraction uses **its own fixed normal map tiling (0.45)**, independent of `normal_tiling`. At low values of that slider the lighting normal would be nearly constant across the surface and the distortion would disappear.



The submerged parts of both piles are shifted and wavering relative to the parts above water — that is `refraction_strength` at work. At the waterline each pile has its own collar of foam.

Waves and the normal map

`wave_height` displaces vertices with two sine waves. It only works on a subdivided mesh — a plane with no subdivisions has nothing to move. The lake in the demo scene is subdivided 44 by 44.

`normal_map` is sampled twice, as two layers scrolling in different directions, and the results are summed. `normal_strength` decides how far the result tilts the normal away from the flat face of the mesh, and `wave_speed` drives the pace of the whole animation, vertex waves included.

The project ships no bitmaps. The normal map and the foam mask are generated from noise inside Godot (Textures/, 512 by 512 pixels, seamless).

Painted zones — vertex colours

Three vertex colour channels drive the shader. An unpainted mesh is white, which means: everything visible, animated and without painted foam.

Channel	Value 0	Value 1 (default)
R (red)	island of foam on the surface	no painted foam
G (green)	pixel discarded — a hole in the surface	water visible
B (blue)	animation frozen	waves, foam and wind running

The green channel is for cutting the water away above cave mouths or under buildings, the blue one for stopping motion in sheltered places, for instance under a bridge or under a sheet of ice. The green threshold is 0.5, so paint black, not grey.

Wazne. The demo scene uses unpainted planes, so none of these three mechanisms show when you run it. To see them you have to paint vertex colours on a mesh of your own.

Parameter groups in the Inspector

Group	What it covers
Colors and Depth	shallow and deep colour, depth reference, band count, shore glow
Lighting and Toon	toon highlight and its flattening, fresnel, refraction strength
Foam and Shore	foam pattern, colour, density, reach, speed, edge break-up, mask hardness, blend mode
Caustics	tint, strength, scale and speed of the veins on the bottom, chromatic aberration
Sparkle	separate layer of flickering points — strength, density, size
Waves Geometry	normal map, its tiling and strength, animation speed, wave height

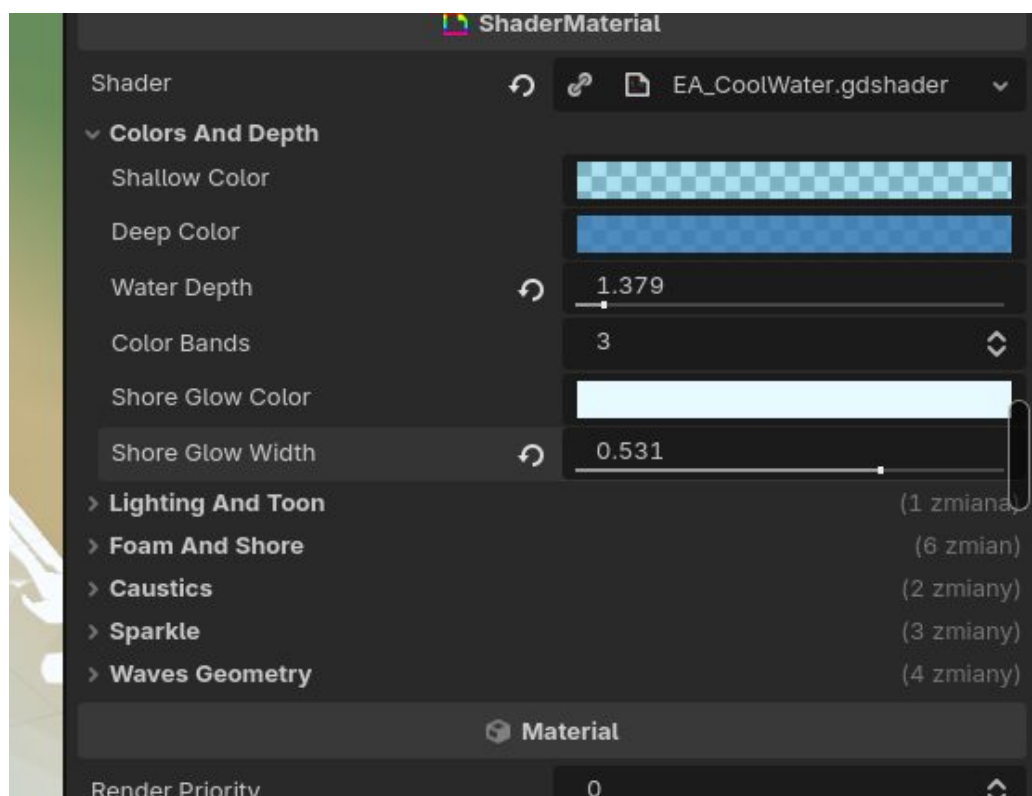
The groups fold independently. The number in brackets next to a collapsed header says how many parameters in that group differ from the shader defaults.

Full parameter list

Everything the shader exposes in the Inspector, group by group. The defaults apply to a new, empty material created on this shader — the materials shipped with the pack have their own values, listed further down.

Colors and Depth

Parameter	What it does	Range	Default
shallow_color	colour and transparency of water over the shallows	colour + alpha	light blue, alpha 0.65
deep_color	colour and transparency of water over the deep end	colour + alpha	mid blue, alpha 0.92
water_depth	ground distance in metres at which the water reaches full deep_color	0.1–20.0	3.0
color_bands	how many steps the way from shallow to deep is split into	1–8	3
shore_glow_color	colour of the band at the shoreline	colour	near white
shore_glow_width	width of that band as a fraction of water_depth	0.0–0.75	0.15



Colors and Depth group in the Inspector

Lighting and Toon

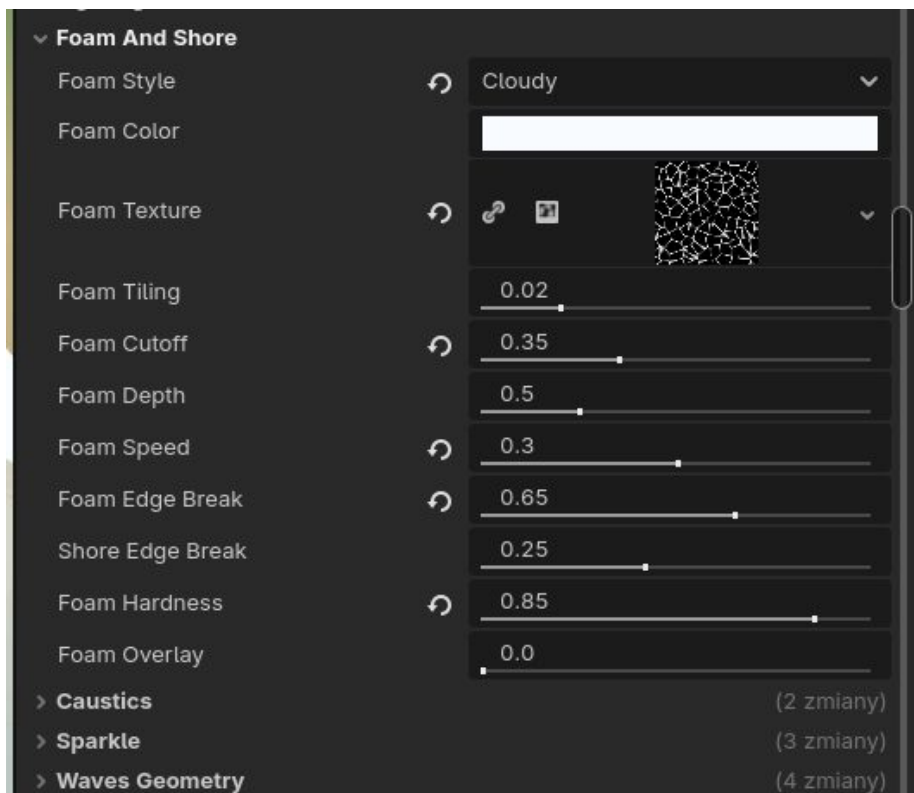
Parameter	What it does	Range	Default
specular_color	highlight colour	colour	white
specular_size	how tight the highlight is — higher narrows the spot	5.0–60.0	24.0
specular_softness	softness of the highlight edge; 0 is a hard cut	0.0–1.0	0.25
specular_hardness	second flattening of what specular_softness left	0.0–1.0	0.85
specular_intensity	highlight brightness	0.0–1.0	0.5
fresnel_power	how much the surface brightens at grazing angles	0.0–3.0	1.0
refraction_strength	strength of the offset applied to what is under the water	0.0–0.1	0.02



Lighting and Toon group in the Inspector

Foam and Shore

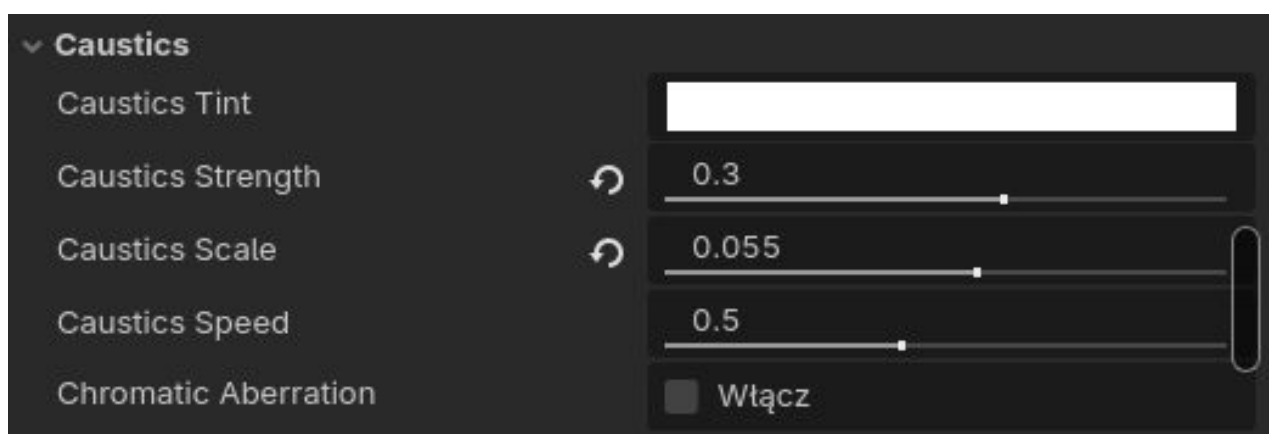
Parameter	What it does	Range	Default
foam_style	foam pattern: Texture, Cloudy, Blobs, Scatter, Cellular	5 modes	Cellular
foam_color	foam colour	colour	white
foam_texture	pattern texture, used in Texture mode only	texture	EA_Foam_Vorono i
foam_tiling	density of the foam pattern	0.0–0.1	0.02
foam_cutoff	how far into the foam_depth range the foam actually reaches	0.0–1.0	0.5
foam_depth	metres from the shore covered by the shoreline foam mechanism	0.0–2.0	0.5
foam_speed	how fast the foam pattern scrolls	0.0–0.6	0.4
foam_edge_break	breaking the foam band into irregular patches	0.0–1.0	0.6
shore_edge_break	disturbance of the waterline itself, in metres	0.0–0.6	0.25
foam_hardness	how abruptly the foam mask ends	0.0–1.0	0.8
foam_overlay	foam blending: 0 flat colour, 1 overlay mode	0.0–1.0	0.0



Foam and Shore group in the Inspector

Caustics

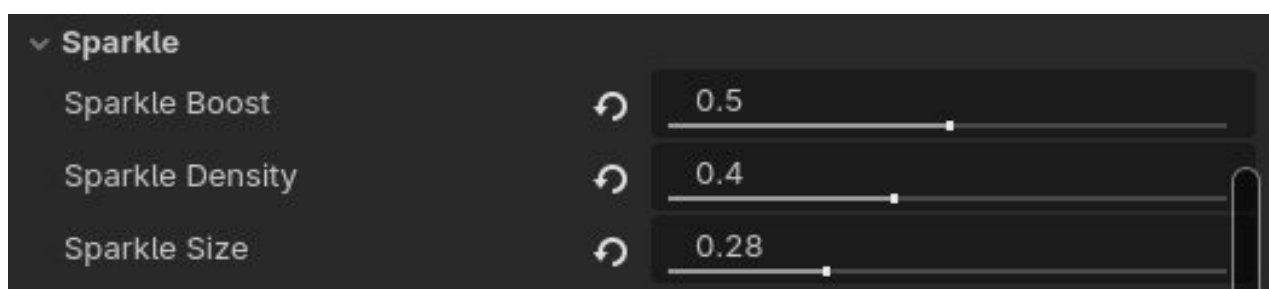
Parameter	What it does	Range	Default
caustics_tint	colour modulating the light veins on the bottom	colour	white
caustics_strength	brightness of the veins	0.0–0.5	0.25
caustics_scale	density of the vein network	0.0–0.1	0.05
caustics_speed	how fast the veins creep across the bottom	0.0–1.2	0.5
chromatic_aberration	rainbow fringe along the veins; costs three times as much	on / off	on



Caustics group in the Inspector

Sparkle

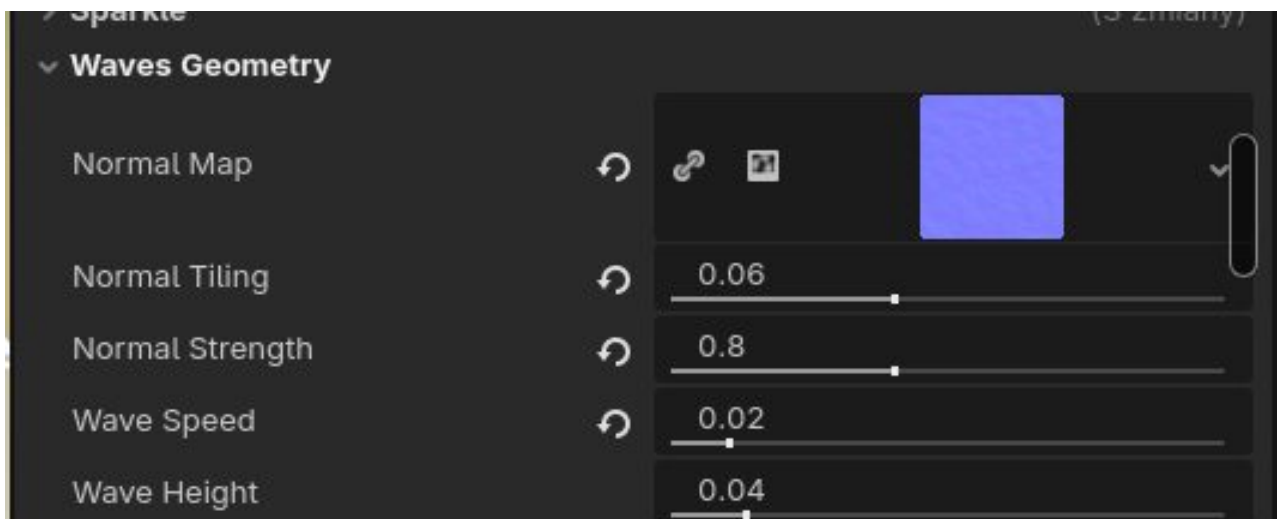
Parameter	What it does	Range	Default
sparkle_boost	brightness of the whole sparkle layer; at 0 the mechanism is skipped	0.0–1.0	0.0
sparkle_density	how many glints appear and how close together	0.0–1.0	0.5
sparkle_size	size of a single glint	0.0–1.0	0.35



Sparkle group in the Inspector

Waves Geometry

Parameter	What it does	Range	Default
normal_map	surface normal map	texture	EA_Water_Normal
normal_tiling	normal map tiling used for lighting	0.0–0.15	0.08
normal_strength	how far the map tilts the normal from the flat mesh face	0.0–2.0	0.5
wave_speed	pace of the whole animation, vertex waves included	0.0–0.2	0.05
wave_height	vertex wave amplitude in metres; needs a subdivided mesh	0.0–0.3	0.04



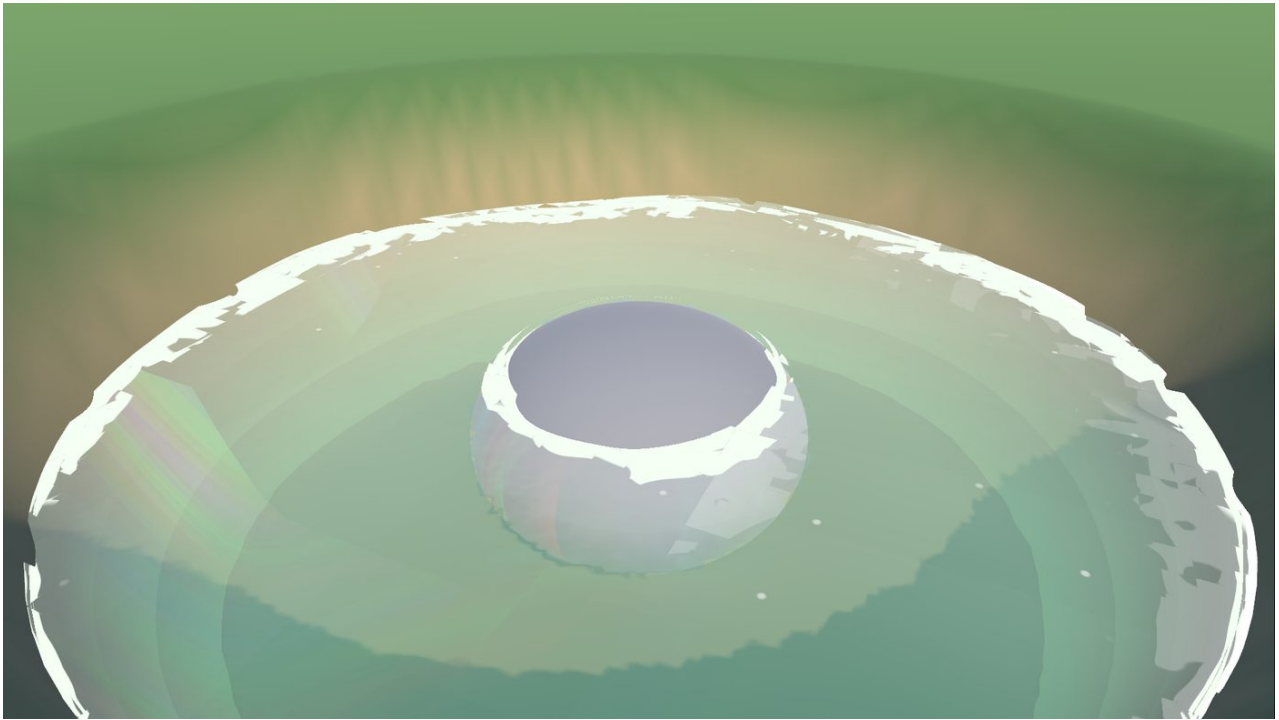
Waves Geometry group in the Inspector

Wazne. refraction_strength uses its own normal map sample at a fixed tiling of 0.45. normal_tiling has no effect on refraction.

The three ready materials

The defaults listed above with each parameter are the state a **new, empty material** created on this shader gets. The three materials shipped with the pack are tuned and override many of those values — which is why the table below shows different numbers. That is a starting point versus a finished preset, not a discrepancy.

Parameter	Tropical	Deep Blue	Lagoon
shallow colour	turquoise	light blue	green
deep colour	mid blue	dark blue	dark green
water_depth	5.42	3.0	2.0
color_bands	3	3	4
shore_glow_width	0.477	0.15	0.12
specular_softness	0.05	0.05	0.05
specular_hardness	0.85	0.85	0.85
foam_style	Cloudy	Cloudy	Cloudy
foam_depth / foam_cutoff	2.0 / 0.30	0.5 / 0.35	0.4 / 0.3
foam_edge_break	0.7	0.65	0.65
shore_edge_break	0.25 (default)	0.25 (default)	0.25 (default)
foam_hardness	0.85	0.85	0.85
foam_overlay	0.0 (default)	0.0 (default)	0.0 (default)
caustics_strength	0.3	0.3	0.35
chromatic_aberration	no	no	yes
sparkle_boost	0.7	0.5	0.55
sparkle_density	0.45	0.4	0.35
sparkle_size	0.3	0.28	0.25
wave_speed	0.001	0.02	0.03



The round pool with the Lagoon material — the only one in the demo with chromatic aberration enabled. The rainbow streaks show on the left edge and around the boulder. The green hue and the narrow shore glow are what set it apart from the other two materials.

Using the shader in your own scene

1. Add a MeshInstance3D with a **subdivided plane** — vertex waves need vertices to act on. 2. Assign one of the three materials from Materials/ as **Material Override**. 3. Place the plane over a dip in the terrain. There has to be geometry underneath — over empty space there is nothing to measure and the water goes flat.

Project structure

Path	What it is
Shaders/EA_CoolWater.gdshader	the shader, commented inside the file
Materials/EA_Water_Tropical.tres	turquoise shore water
Materials/EA_Water_DeepBlue.tres	cool pool blue
Materials/EA_Water_Lagoon.tres	green pond
Materials/EA_Terrain.tres	terrain material, vertex colours
Meshes/EA_Terrain.res	demo terrain mesh, ready made
Textures/EA_Water_Normal.tres	normal map generated from noise
Textures/EA_Foam_Voronoi.tres	foam mask, voronoi cells
Scripts/DemoCamera.gd	orbit camera
Scripts/UI_ParameterPanel.gd	slider panel logic
Scenes/WaterDemo.tscn	demo scene
Scenes/UI_ParameterPanel.tscn	slider panel, layout saved in the scene

The terrain is a saved mesh and is not generated when the game starts. The scene looks the same in the editor as it does when running. It holds three basins: a lake with a gentle beach, a round pool with steep walls and a rectangular pool.

What to leave alone

The renderer. Forward+ or Mobile. Compatibility does not provide the form of the depth buffer this shader needs.

The scene tonemap. The water carries its colour through emission on a black albedo, so that ambient light does not tint it a second time. A filmic tonemap therefore flattens the saturation to pastel — the demo scene uses linear.

Uniform names. Materials bind values by name. Rename one and the value disappears without warning, and has to be set again.

Overlapping surfaces. The shader does not write depth (`depth_draw_never`) so that transparency works correctly. Two surfaces stacked one above the other will bleed through each other.

The scale of the foam texture. The reach of the foam tongues grows with the brightness of the pattern sample. Swapping `foam_texture` for an image with thick white areas pushes the foam far past the shore, all the way into the middle of the basin. The pattern is meant to be a thin web on a dark background.

Where the shader comes from

The same shader draws the water in the **Roadside Tales: Backroad Bridges** pack, there in a Unity URP version (EmaceArt/EA_CoolWater). The version in this pack is rewritten in GDScript and keeps the behaviour of that one: colour bands, shore glow, foam, caustics and toon specular. One difference concerns lighting — in Unity the highlight only took the main directional light into account, here it

responds to every light in the scene.

The Tropical material started from values carried over one to one from the Water_Experimental material in that pack, with the colours converted from linear space to sRGB. It has moved on from that state in a few places: fresnel_power raised from 1.02 to 1.6 (at the lower value the surface went white when viewed from above), the foam style changed from Texture to Cloudy, foam_cutoff lowered from 0.647 to 0.30, chromatic aberration switched off and caustics_strength lowered from 0.47 to 0.30 — the old surface looked too photoreal.

What has not been tested

The order of parameters in the Inspector has not been confirmed in an open editor window. The groups should match, since they follow the uniform declarations in the shader, but the panel itself has not been checked live.

The painted vertex colour zones are carried over from the Unity version. The demo scene does not use them, so they have not been seen working in Godot.

The Mobile renderer has not been tested. It uses the same depth and screen textures as Forward+, so it should work.

Join us on Discord

This pack keeps growing, and the next versions come out of what the people using it report back. On the server you can show how the water looks in your own scene, ask about settings, report a fault or request a parameter that is missing.

<https://discord.com/invite/ctXaf5Ftqw>

— **EmacEArt**